任务四参考学习资料

这份资料是完成第四章任务的**学习路线**，不是需要从头到尾看完的课程清单。先跑一遍任务项目、读一遍 `故障工单.md`，再按下面的顺序查缺补漏。已经会的部分可以跳过，卡住时回到对应小节。具体提交要求以任务书为准。

学完要能做什么

- 拿到一个跑不通或结果不对的项目，能先复现现象，再把范围缩到一个文件、一个函数、一行逻辑；
- 说清相机内参、畸变系数、靶标坐标系和机械臂基坐标系之间的关系；
- 用 `solvePnP` 从四个点的二维/三维对应解出位姿，并知道点序、单位、畸变参数为什么会把它带偏；
- 用重投影误差和已知尺寸检查结果是否合理，并知道重投影小**不代表**位姿对；
- 把结论写成别人能复核的排查记录，而不是“改完就好了”。

开始前确认你能运行 Python 和 OpenCV，并知道当前用的是哪个环境；环境准备可回看第 0 章教程，这里不重复。

推荐学习顺序

1. 先把问题复现出来

先做： 按 `README.md` 跑一遍 `python main.py`，把终端输出、报错、`output.avi`、`results.json` 都保留下来。报错不要只看最后一行，从下往上看：最后一段是 OpenCV 内部断言，往上几行才是自己代码里出错的位置。

动手： 用 `data/frames/` 里的单帧写一个最小脚本，只调用 `detector.py`，把检测到的矩形、轮廓点和中间量打印出来。单帧调试比逐帧跑视频快得多。

自检： 能说出“这条工单在哪个输入上、第几帧、看到什么”。说不清就先别改代码。

调试的几个基本手段：

- **单变量**：一次只改一个因素，改完立刻对照同一输入；
- **最小复现**：把问题缩到一张图、一个函数、一个数值；
- **二分**：在流程中间插打印/断言，判断问题在“前面”还是“后面”；
- **对照**：用一个已知正确的量（靶标边长、桌面 z=0、样例帧输出）判断偏了多少、往哪个方向偏；
- **看历史**：项目里如果有 git 记录，`git diff`、`git log` 常常能直接指出哪次改动引入了问题。

2. 相机模型、内参与畸变

先看： OpenCV 官方：相机标定 的前半部分，重点看内参矩阵 `K` 的每一项含义。

针孔模型下，相机坐标系中的点 `(X, Y, Z)` 投到图像：

```
u = fx * X / Z + cx
v = fy * Y / Z + cy
```

`fx, fy` 是焦距（像素单位），`cx, cy` 是主点。真实镜头还有径向和切向畸变，所以 OpenCV 的投影函数都要同时传 `cameraMatrix` 和 `distCoeffs`。**同一个位姿问题里，求解和画图必须使用同一套内参和畸变参数**，否则画面中心看不出来、画面边缘偏差很大。

自检： 能回答“`tvec` 的单位是什么”“畸变系数不传会怎样”“为什么画面边缘对畸变更敏感”。

3. 靶标检测与角点顺序

靶标是黑色正方形，检测部分已经写好：灰度化 → 反向阈值 → 找外部轮廓 → 四边形筛选 → 最小外接矩形。**角点怎么从矩形里取、四个点按什么顺序排、要不要做亚像素细化，都是你要决定的。**

关键在**顺序**。四个点没有名字，是人给它们定的名字。本项目约定：

```
TL(-h,+h)   TR(+h,+h)
BL(-h,-h)   BR(+h,-h)      h = 边长/2，单位毫米
```

图像里的左上、右上、右下、左下必须和三维点表的第 1、2、3、4 行一一对应。顺序错一位，解出来的位姿就会整体旋转或镜像；而正方形靶标旋转 90° 后看起来和原来一样，所以**画面可能完全正常，偏航角却差了 90°。**

自检： 在图上按顺序标出 0、1、2、3，确认和你自己写的三维点表顺序一致；把同一个角点单独画出来，看它是否落在预期位置。

4. solvePnP：二维与三维的对应

先看： OpenCV 官方：solvePnP 文档 中 `solvePnP` 的参数说明，以及 `SOLVEPNP_IPPE` 的适用场景（平面四点）。

`solvePnP` 输入四组“三维点 ↔ 二维像素点”的对应，输出靶标坐标系到相机坐标系的旋转向量和平移向量。它只保证**这几组对应在几何上尽量自洽**，不保证你给的点序、单位、内参是对的。

几个最容易出问题的地方：

- **单位**：三维点用毫米，内参按像素，`tvec` 就是毫米；三维点用米，`tvec` 就是米。混用会让结果差 1000 倍，而且程序不会报错。
- **点序**：见上一节。平面四点有旋转对称性，点序错可能得到一个“重投影很好但姿态错”的解。
- **畸变**：传 `None` 等于忽略畸变，边缘帧会偏。
- **方法**：四点共面优先用 `SOLVEPNP_IPPE`；`SOLVEPNP_ITERATIVE` 在平面四点上不一定稳。

自检： 把同一个位姿分别用检测到的角点和重投影点代入，结果应基本一致；把靶标边长改 10%，`tvec` 应同比例变化。

5. 手眼变换与坐标系

相机看到的位姿在相机坐标系里，机械臂要的是基坐标系。两者由一个固定的 4×4 齐次矩阵 `T_base_cam` 联系：

```
P_base = T_base_cam @ P_cam
T_base_target = T_base_cam @ T_cam_target
```

- 矩阵乘法**顺序不能交换**，`T_base_cam @ T_cam_target` 和 `T_cam_target @ T_base_cam` 结果完全不同；
- 求逆的方向也要分清：`T_base_cam` 是“相机 → 基座”，它的逆是“基座 → 相机”；
- 齐次矩阵的平移单位要一致，旋转部分应满足正交性（`R @ R.T ≈ I`）。

自检： 用靶标平放在桌面（`z=0`）这个已知条件验证：算出来的基坐标 `z` 应该接近 0。偏得很远，多半是变换方向或单位不对。

6. 重投影验证与它的盲区

把三维角点用解出的 `rvec/tvec` 投回图像，与检测到的角点比较，就是重投影误差。它是很好的自洽检查，但有两个盲区：

- **重投影小 ≠ 位姿对**：对称靶标点序错误时，重投影可以只有 0.1px，偏航角却差 90°；
- **不能用它发现单位错误**：单位整体缩放时，图像投影仍然吻合。

所以还要用**独立信息**交叉验证：靶标边长、桌面 `z=0`、样例帧的坐标/偏航角、卷尺实测距离。

自检： 能解释“为什么重投影很好、位姿却是错的”。

7. 静默错误怎么缩小范围

先分清两类问题：

- **会报错的**：顺着“这个报错的输入是从哪里来的”往上找，而不是停在库内部的断言上；
- **不报错但结果错的**：按下面四个问题逐层缩小。

1. **数量级对吗？** 把结果与一个已知量比较（靶标边长、相机到桌面的距离、样例数值）。差 10、100、1000 倍，先查单位和坐标系约定，不要急着改公式。
2. **误差是整体一致，还是局部放大？** 整体一致，查坐标变换链的方向和乘法顺序；只在画面边缘变大，查畸变、标定和图像尺寸。
3. **位置和姿态分别对不对？** 位置对、姿态错，查二维/三维对应关系与点序；姿态对、位置错，查变换方向和平移单位；两者都错但重投影很小，很可能拿到了一个“自洽但语义错误”的解。
4. **和帧序有关吗？** 只有部分帧错、或上一帧结果残留，查状态、缓存和初始化。

报错类问题还可以先看三点：报错位置在库内部时，先看传给它的参数从哪里读来；文件/视频类操作不报错但产物不对时，先核对创建参数与写入内容的尺寸、类型和路径；数值在阈值内但结论不对时，用独立信息交叉验证，而不是只看误差指标。

每做一次判断，都用同一帧、同一输入做前后对照，并在排查记录里写下你**排除了什么**。工单不保证与上面的分类一一对应，也不保证一条只属于一类；这张清单是缩小范围的方法，不是答案索引。

8. 加分项怎么入手

- **亚像素**：`cv2.cornerSubPix` 需要灰度图和角点初值，窗口 5～7、迭代 20～30 次即可；对比开启前后同一帧的重投影和相邻帧位姿差。
- **自主标定**：`findChessboardCorners(gray, (9, 6))` + `cornerSubPix` + `calibrateCamera`；用标定结果的 `RMS` 判断质量，再和给定内参比较 `fx/fy/cx/cy`。
- **稳定输出**：可以用简单的滑动平均/限幅，也可以对丢失帧做“保持 + 标记 + 恢复”策略；要用数字说明改进了多少，而不是“看起来更稳”。

查阅入口与自检

- OpenCV 官方 Python 教程入口：查函数用法；
- calib3d 模块索引：查 `solvePnP`、`projectPoints`、`calibrateCamera` 的参数；
- 第三方文章和视频用于入门；函数签名、返回值和版本差异以当前安装的 OpenCV 文档与实际运行结果为准。

完成基础任务后，试着不看代码回答：靶标坐标系和基坐标系分别怎么定义？`tvec` 和 `base` 有什么区别？重投影误差为什么不能单独证明位姿正确？如果把靶标边长写成 100mm，结果会怎么变？答不上来就回到对应小节，用单帧和打印再验证一次。

求助格式

写清目标、环境、运行命令、预期与实际、完整报错或结果、已经尝试过的检查，附必要代码和文件。这样同学和指导者才能针对真正卡住的环节帮你定位，而不是只看到一个“跑不出来”。